How I Would Automate This Mobile App

In this section I want to explain, in plain terms, how I would build the automation for this app and how it would grow over time. The framework in this project is a small, working version of exactly what I describe here, so it is not just theory.

Which tools I would choose, and why

My main choice would be **WebdriverIO together with Appium, written in TypeScript**.

Appium is the industry standard for mobile testing. The big advantage is that one set of tests can drive both iOS and Android, instead of writing and maintaining two separate test suites. That matters a lot when the same product ships on both platforms.

WebdriverIO is the tool that actually runs the tests and ties everything together. It works very naturally with TypeScript and with Appium, it can run tests in parallel, retry flaky ones, connect to device clouds, and produce good reports. It is also widely used in the industry, which means it is easy to hire people who already know it.

I chose TypeScript because it catches a lot of small mistakes while I am writing the tests, before they ever run. As the test suite grows, that safety makes the whole thing much easier to maintain and change.

A quick word on the native tools (Apple's XCUITest and Google's Espresso). They are excellent, and I would still use them, but for a different job. They are best for fast, low level tests written by the app developers themselves. For the end to end user journeys that QA owns, I prefer Appium and WebdriverIO because one suite covers both platforms. So rather than choosing one over the other, I would use them together, each for what it does best.

How I would structure the framework

The main idea is that a test should be written once and then run on web, iOS, and Android, with only the underlying details changing per platform. Here is how the project is laid out:

```
config/        wdio.shared.conf.ts        # the settings shared by every platform
               wdio.web.conf.ts           # runnable today: mobile-emulated Chrome
               wdio.android.conf.ts       # Appium for Android
               wdio.ios.conf.ts           # Appium for iOS
test/
  specs/       *.e2e.ts                   # the actual tests: business steps and checks
  pageobjects/ *.page.ts                  # one file per screen, holding the actions
  selectors/   index.ts                   # one map of how to find each element per pla
```

```
data/       users.ts                  # generates fresh test data for each run
support/    platform.ts               # knows which platform is running
```

The decisions behind this:

- **Page Objects.** Each screen has its own file that describes what you can do on it, such as "log in" or "add to cart". The tests themselves just say what the user does, in plain steps. This means that if a screen changes, I only fix it in one place.
- **One place for element locations.** Every element knows how to be found on web, Android, and iOS. For the native apps I rely on accessibility identifiers, which are the most stable way to find things and also help users who rely on screen readers. When the real app is built, this is the only file that needs to change.
- **Tests that do not give up easily.** There is a shared helper that waits for an element, scrolls to it, and tries an alternative way to tap it if something gets in the way. On the public site, ads literally sat on top of buttons and blocked them, so this kept the tests reliable.
- **Fresh data every time.** Each run creates a brand new user, so the registration tests never clash with an account that already exists.
- **Tests that do not depend on each other.** State is cleared before each test, so the order they run in never matters.

This keeps things small but ready to grow. Adding a new screen means one new Page Object and one entry in the locator map. Adding a new platform means one new config file.

How the tests would run on iOS and Android

- **On my own machine while developing**, I would use Appium with simulators (iOS) and emulators (Android). The framework is already set up for this; it just needs the Apple and Android development tools installed, plus a built version of the app to point it at.
- **The app itself** comes from the app team's own build process as a file (an `.apk` for Android, an `.app` or `.ipa` for iOS). The automation never builds the app, it just runs against it.
- **For wide device coverage**, I would use a device cloud such as BrowserStack, Sauce Labs, or Firebase Test Lab. These let me run on many real phones and operating system versions without having to own a room full of devices. Switching to them does not require changing the tests.
- **The web version** (`npm run test:web`) runs the same tests against the responsive website in a mobile sized Chrome window. It only needs Node and Chrome, runs in seconds, and is the part that runs successfully in this project today. It is a fast way to prove the tests and Page Objects work without needing any phone or emulator.

How this fits with CI/CD

I would set this up so the cheap, fast checks run all the time, and the heavier, broader checks run on a schedule.

- **On every code change (pull request):** run the type check and the fast web tests. This takes minutes, needs no devices, and stops broken changes from being merged.

- **After merging, and overnight:** run the full Appium tests on a device cloud, across a sensible set of iOS and Android devices, once the latest app build is ready.
- **Before a release:** run a wider set of devices plus the full checkout journey, as a final gate.
- **Reporting:** every run produces a clear report, with screenshots and video saved when something fails, so problems are quick to understand. Genuinely flaky tests are quarantined and retried so they do not hide real failures.

In short

I would use WebdriverIO with Appium in TypeScript for the cross platform user journeys, and lean on the native tools for the developers' lower level tests. I would build everything around Page Objects and a single map of element locations, so tests are written once and run everywhere. I would run them on simulators and emulators locally, on a device cloud for breadth, and gate every pull request with the fast web tests. The project here already does the core of this and runs successfully today.